

UNIVERZA ALMA MATER EUROPAEA

Evropski center, Maribor

Izpitna naloga pri predmetu: Odprta koda

**Android aplikacija
(CENE GORIVA PO DRŽAVAH)**

Študent: Marko Tojagić

Nosilec predmeta: prof. Dušan Fugina

Letnik študija: 3. letnik

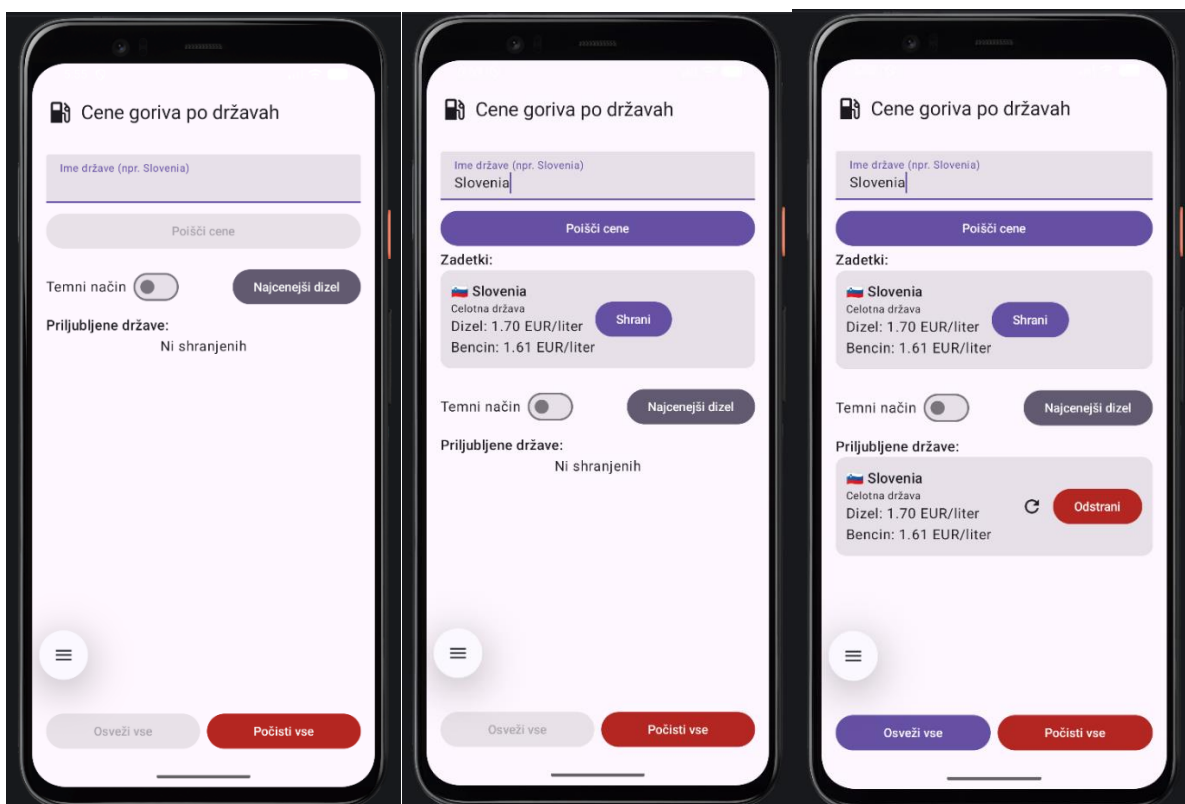
Študijski program: SPLETNE IN INFORMACIJSKE TEHNOLOGIJE

Elektronska pošta: marko.tojagic@almamater.si

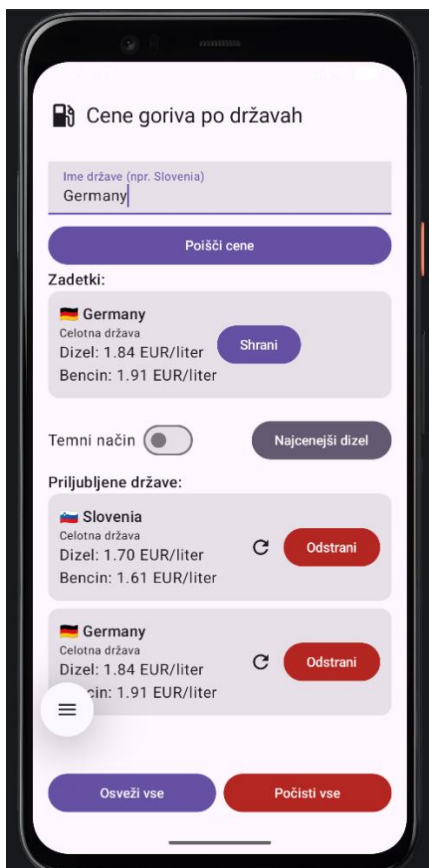
Maribor, junij 2026

Opis delovanja aplikacije

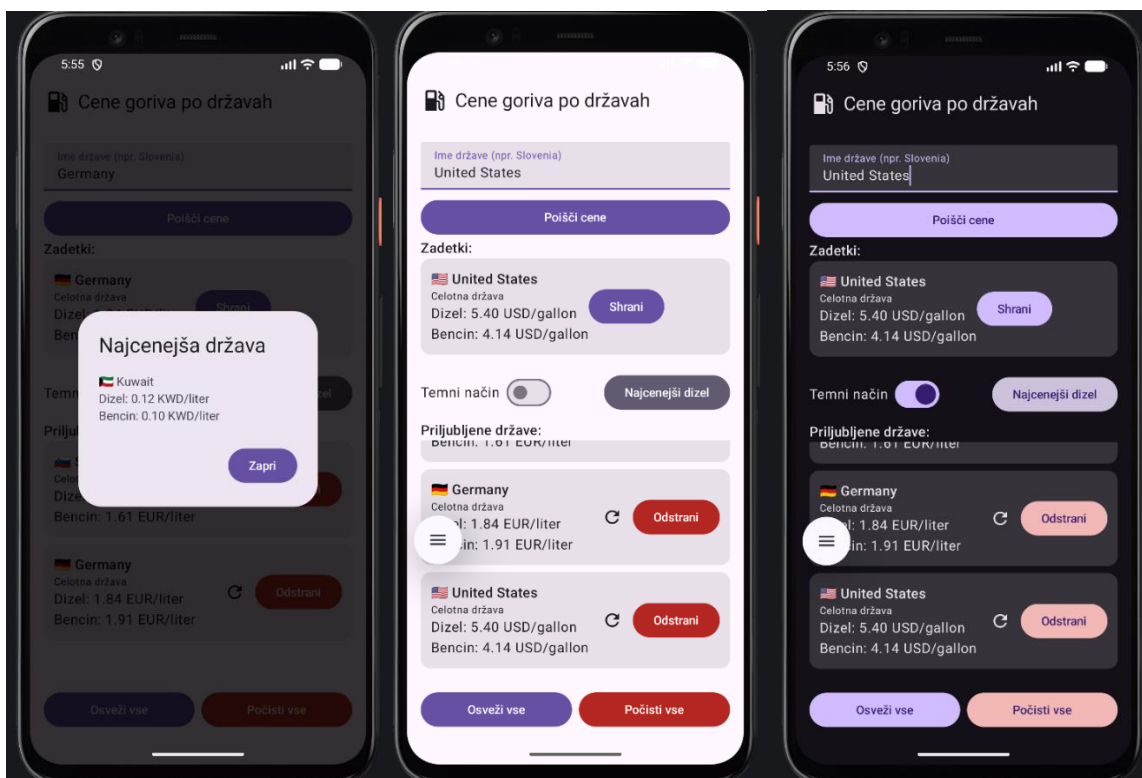
Aplikacija uporabniku omogoča, da hitro poišče trenutne cene dizla in bencina za poljubno državo na svetu. V zgornje vnosno polje uporabnik vpiše ime države, na primer **Slovenia** ali **United States**, nato klikne gumb **Poišči cene**. Aplikacija preko brezplačnega **OpenVan API** pridobi podatke o cenah goriva za vpisano državo in jih prikaže v seznamu **Zadetki**. Pri vsakem zadetku so poleg imena države prikazani zastavica, cena dizla in bencina v originalni valuti ter enota (liter ali gallon). **Primer: Dizel: 1,70 EUR/liter ali Dizel: 5,40 USD/gallon**. Ko uporabnik klikne gumb **Shrani** ob želeni državi, aplikacija državo shrani v lokalno bazo **SharedPreferences** s pomočjo **JSON (knjižnica Gson)**. Če poskuša shraniti že obstoječo državo, ga aplikacija opozori z obvestilom. Shranjene države se takoj prikažejo v spodnjem seznamu **Priljubljene države**.



Drugi del zaslona je namenjen upravljanju s shranjenimi državami. Vsaka shranjena država ima poleg podatkov o cenah dva gumba. Z **ikono za osvežitev** uporabnik posodobi cene samo za to državo, z gumbom **Odstrani** pa jo izbriše iz seznama in iz **SharedPreferences**. Na dnu zaslona sta še dva gumba: **Osveži vse** in **Počisti vse**. Prvi posodobi cene za **vse** shranjene države hkrati, drugi pa z enim klikom izbriše vse priljubljene države. Vsako dejanje je potrjeno s kratkim sporočilom.



Aplikacija ponuja tudi dve dodatni funkcionalnosti. S stikalom **Temni način** uporabnik preklopi med svetlo in temno barvno shemo celotnega vmesnika. Z gumbom Najcenejši dizel pa aplikacija med vsemi državami sveta poišče tisto z najnižjo ceno dizla in rezultat prikaže v pogovornem oknu s podatki o državi in cenami. Vse podatke o cenah goriva aplikacija vedno pridobiva neposredno iz API-ja v živo, zato so vedno sveži. Shranjevanje priljubljenih držav omogoča, da uporabnik ob naslednjem zagonu aplikacije takoj vidi svoj seznam brez ponovnega iskanja. Aplikacija je v celoti razvita v Kotlinu z Jetpack Compose in uporablja korutine za asinhrono komunikacijo z API-jem.



Opis za Model.kt datoteko

Datoteka **Model.kt** vsebuje podatkovne razrede, ki določajo obliko podatkov, s katerimi aplikacija dela. Prvi razred je **Bencinska**, ki se uporablja za podatke, pridobljene iz API-ja. Vsebuje polja za ime države, kodo države, valuto, enoto (liter ali galon), ceno dizla in ceno bencina. Ceni sta lahko ničelni vrednosti, ker nekatere države ne poročajo obeh cen. Drugi razred je **Shranjena**, ki je skoraj enak, vendar se uporablja za shranjevanje v **SharedPreferences**. Podatkovna razreda sta deklarirana s ključno besedo **data**, kar pomeni, da samodejno pridobita metode za primerjanje, kopiranje in izpis. V isti datoteki je definirana tudi pomožna funkcija **kodaVZastavo**. Ta funkcija prejme kodo države, na primer "SI", in jo pretvori v Unicode emoji zastavice. Če koda ni dolga dva znaka, vrne nevtralno zastavico. Funkcija izračuna ustrezna Unicode odstopanja glede na črki 'A' in ju združi v niz. Zastavica se nato prikazuje poleg imena države v uporabniškem vmesniku.

```
1 package com.example.izpitna
2
3 8 Usages
4 data class Bencinska(
5     val ime: String,
6     val koda: String,
7     val valuta: String,
8     val enota: String,
9     val cenaDizla: Double?,
10    val cenaBencina: Double?
11 )
12
13 6 Usages
14 data class Shranjena(
15     val ime: String,
16     val koda: String,
17     val valuta: String,
18     val enota: String,
19     val cenaDizla: Double?,
20     val cenaBencina: Double?
21 )
22
23 3 Usages
24 fun kodaVZastavo(koda: String): String {
25     if (koda.length != 2) return " "
26     val first = 0x1F1E6 + (koda[0].uppercaseChar() - 'A')
27     val second = 0x1F1E6 + (koda[1].uppercaseChar() - 'A')
28     return String( codePoints = intArrayOf(first, second), offset = 0, length = 2)
29 }
```

Opis za Api.kt datoteko

Datoteka **Api.kt** skrbi za vso komunikacijo s svetom zunaj aplikacije. Vsebuje dve asinhroni funkciji, označeni s ključno besedo **suspend**. Funkcija **dobiVse** pokliče javni API na naslovu **https://openvan.camp/api/fuel/prices** in pridobi seznam vseh držav s cenami goriva. Klic poteka znotraj bloka **withContext(Dispatchers.IO)**, kar zagotovi, da se omrežna operacija izvede v ozadju in ne blokira glavne niti uporabniškega vmesnika. Funkcija ustvari povezavo s strežnikom, nastavi časovni omejitvi za vzpostavitev povezave in branje, prebere odgovor kot tekst in povezavo zapre. Odgovor je v JSON obliki. S pomočjo knjižnice Gson se celoten odgovor pretvori v zemljevid. Iz zemljevida se izlušči objekt **data**, ki vsebuje seznam držav. Za vsako državo se preberejo ime, koda, valuta, enota ter ceni dizla in bencina, nato pa se ustvari objekt **Bencinska**. Če med izvajanjem pride do izjeme, na primer zaradi pomanjkanja internetne povezave, se izjema ujame in funkcija vrne prazen seznam, s čimer se aplikacija ne sesede. Druga **suspend** funkcija je **dobiCene**, ki sprejme ime države, pokliče **dobiVse** in nato iz dobljenega seznama izloči samo tiste države, katerih ime se ujema z iskanim nizom, pri tem ne razlikuje med velikimi in malimi črkami. Tretji del datoteke predstavljata funkciji za shranjevanje v **SharedPreferences**. **shraniPriljubljene** prejme kontekst in seznam objektov **Shranjena**, ga s pomočjo Gson pretvori v JSON niz in shrani pod ključem **priljubljene** v zasebno shrambo aplikacije, imenovano **moja_app**. **naloziPriljubljene** pa ta niz prebere, ga pretvori nazaj v seznam in vrne. Če niz ne obstaja, vrne prazen seznam. Za pravilno pretvorbo se uporabi **TypeToken**, ki Gsonu pove, da pričakuje seznam objektov **Shranjena**.

MainActivity.kt

API.kt

MODEL.kt

build.gradle.kts (:app)

AndroidManifest.xml

```

1 package com.example.izpitna
2
3 import android.content.Context
4 import com.google.gson.Gson
5 import com.google.gson.reflect.TypeToken
6 import kotlinx.coroutines.Dispatchers
7 import kotlinx.coroutines.withContext
8 import java.net.HttpURLConnection
9 import java.net.URL
10
11 suspend fun dobiCene(drzava: String): List<Bencinska> = withContext(Dispatchers.IO) {
12     dobiVse().filter { it.ime.equals(other = drzava, ignoreCase = true) }
13 }
14
15 4 Usages
16 suspend fun dobiVse(): List<Bencinska> = withContext(Dispatchers.IO) {
17     val url = "https://openvan.camp/api/fuel/prices"
18     return@withContext try {
19         val conn = URL(spec = url).openConnection() as HttpURLConnection
20         conn.requestMethod = "GET"
21         conn.connectTimeout = 5000
22         conn.readTimeout = 5000
23         val odgovor = conn.inputStream.bufferedReader().readText()
24         conn.disconnect()
25
26         val gson = Gson()
27         val tip = object : TypeToken<Map<String, Any>>() {}.type
28         val odgovorMap: Map<String, Any> = gson.fromJson(odgovor, typeOf<tip>())
29         val data = odgovorMap["data"] as? Map<*, *> ?: return@withContext emptyList()
30
31         data.mapNotNull { (_, value) ->
32             if (value is Map<*, *>) {
33                 val cene = value["prices"] as? Map<*, *>
34                 val enota = value["unit"] as? String ?: "liter"
35                 Bencinska(
36                     ime = value["country_name"] as? String ?: return@mapNotNull null,
37                     koda = value["country_code"] as? String ?: "",
38                     valuta = value["currency"] as? String ?: "EUR",
39                     enota = enota,
40                     cenaDizla = (cene?.get("diesel") as? Number)?.toDouble(),
41                     cenaBencina = (cene?.get("gasoline") as? Number)?.toDouble()
42                 ) else null
43             }
44         } catch (_: Exception) {
45             emptyList()
46         }
47     }
48
49 5 Usages
50 fun shраниPriljubljeni(kon: Context, sez: List<Shranjena>) {
51     val prefs = kon.getSharedPreferences(name = "moja_app", mode = Context.MODE_PRIVATE)
52     val json = Gson().toJson(src = sez)
53     prefs.edit().putString(key = "priljubljene", value = json).apply()
54 }
55
56 2 Usages
57 fun naloziPriljubljeni(kon: Context): List<Shranjena> {
58     val prefs = kon.getSharedPreferences(name = "moja_app", mode = Context.MODE_PRIVATE)
59     val json = prefs.getString(key = "priljubljene", defValue = null) ?: return emptyList()
60     val tip = object : TypeToken<List<Shranjena>>() {}.type
61     return Gson().fromJson(json, typeOf<tip>())
62 }

```

Opis za MainActivity.kt datoteko

Datoteka **MainActivity.kt** je jedro aplikacije, saj vsebuje uporabniški vmesnik in vso logiko, ki povezuje uporabnikova dejanja s podatki. Razred **MainActivity** podeduje **ComponentActivity** in v metodi **onCreate** nastavi vsebino zaslona. Najprej definira stanje za temni način, nato pa znotraj **MaterialTheme** glede na to stanje izbere svetlo ali temno barvno shemo in prikliče sestavni del **Aplikacija**. Sestavni del **Aplikacija** vsebuje vsa stanja: besedilo v iskalnem polju, seznam rezultatov iskanja, seznam priljubljenih držav, zastavico, ali se podatki nalagajo, ali poteka osveževanje vseh in ali je bilo iskanje že izvedeno. Prav tako hrani stanji za dialog najcenejše države in sam izračun. Z **LaunchedEffect** se ob prvem zagonu naložijo shranjene države iz **SharedPreferences**. Uporabniški vmesnik je zgrajen s pomočjo **Scaffold**, ki zagotavlja zgornjo vrstico, znotraj nje pa je **Column** z vsemi elementi. Najprej je **TextField** za vnos imena države, ki ob vsaki spremembi ponastavi zastavico **iskanjeIzvedeno**, da se napaka ne prikaže prehitro. Sledi gumb **Poišči cene**, ki ob kliku preveri, ali je vnos neprazen, nato pa v korutini nastavi **nalaganje** in **iskanjeIzvedeno** ter pokliče **dobiCene**. Med nalaganjem se prikaže **CircularProgressIndicator** znotraj **Box**-a. Ko je iskanje končano, se prikažejo rezultati v **LazyColumn** ali pa sporočilo, da država ni najdena. Vsak rezultat je predstavljen s sestavnim delom **Zadetek**, ki prikaže podatke in gumb **Shrani**. Ob shranjevanju se preveri, ali država že obstaja v seznamu priljubljenih; če ne, se doda in shrani, če pa že obstaja, se prikaže obvestilo. Pod rezultati je vrstica s stikalom za temni način in gumbom **Najcenejši dizel**. Sledi **Column** z **Modifier.weight(1f)**, ki zavzame ves preostali prostor in vsebuje seznam priljubljenih držav. Vsaka priljubljena država je predstavljena s sestavnim delom **Priljubljena**, ki vsebuje podatke, gumb za odstranitev in ikono za osvežitev. Na dnu zaslona sta v vodoravni vrsti gumba **Osveži vse** in **Počisti vse**. Prvi ob kliku zažene osveževanje vseh držav, drugi pa izprazni celoten seznam in ga shrani. Vsako dejanje je opremljeno z obvestilom **Toast**. Ko uporabnik klikne **Najcenejši dizel**, se v korutini pokliče funkcija **najcenejsaDrzavaDizel**, ki med vsemi državami sveta poišče tisto z najnižjo ceno dizla. Nato se prikaže **AlertDialog** z izpisanimi podatki. Sestavni deli **Zadetek**, **Priljubljena** in **Stolpec** so napisani ločeno za večkratno uporabo.

Stolpec z uporabo **when** prepozna, ali prejme objekt tipa **Bencinska** ali **Shranjena**, in v obeh primerih izpiše ustrezno oblikovane podatke z zastavico, imenom, ceno in enoto.

```
1 package com.example.izpitna
2
3 import android.os.Bundle
4 import android.widget.Toast
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.foundation.lazy.LazyColumn
9 import androidx.compose.foundation.lazy.items
10 import androidx.compose.material.icons.Icons
11 import androidx.compose.material.icons.filled.LocalGasStation
12 import androidx.compose.material.icons.filled.Refresh
13 import androidx.compose.material3.*
14 import androidx.compose.runtime.*
15 import androidx.compose.ui.Alignment
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.platform.LocalContext
18 import androidx.compose.ui.unit.dp
19 import kotlinx.coroutines.launch
20
21 Usage
22 class MainActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25         setContent {
26             var temniNacin by remember { mutableStateOf( value = false) }
27             MaterialTheme(
28                 colorScheme = if (temniNacin) darkColorScheme() else lightColorScheme()
29             ) {
30                 Aplikacija(temniNacin = temniNacin, onTemniNacinChange = { temniNacin = it })
31             }
32         }
33     }
34
35     Usage
36     @OptIn( ...markerClass = ExperimentalMaterial3Api::class)
37     @Composable
38     fun Aplikacija(temniNacin: Boolean, onTemniNacinChange: (Boolean) -> Unit) {
39         val kon = LocalContext.current
40         var iskalniNiz by remember { mutableStateOf( value = "") }
41         var rezultati by remember { mutableStateOf<List<Bencinska>>( value = emptyList()) }
42         var priljubljene by remember { mutableStateOf( value = naloziPriljubljene(kon)) }
43         var nalaganje by remember { mutableStateOf( value = false) }
```

```

42 var osvezevanjeVseh by remember { mutableStateOf( value = false) }
43 var iskanjeIzvedeno by remember { mutableStateOf( value = false) }
44 val obseg = rememberCoroutineScope()
45 var prikaziDialogNajcenejsa by remember { mutableStateOf( value = false) }
46 var najcenejsaDrzava by remember { mutableStateOf<Bencinska?>( value = null) }
47
48 LaunchedEffect( key1 = Unit) {
49     priljubljene = naloziPriljubljene(kon)
50 }
51
52 Scaffold(
53     topBar = {
54         TopAppBar(
55             title = {
56                 Row(verticalAlignment = Alignment.CenterVertically) {
57                     Icon( imageVector = Icons.Default.LocalGasStation, contentDescription = null, Modifier.size(32.dp))
58                     Spacer( Modifier.width(8.dp))
59                     Text("Cene goriva po državah")
60                 }
61             }
62         )
63     }
64 ) { notranjiRob ->
65     Column(
66         Modifier
67             .padding( paddingValues = notranjiRob)
68             .fillMaxSize()
69             .padding( all = 16.dp)
70     ) {
71         TextField(
72             value = iskalniNiz,
73             onChange = {
74                 iskalniNiz = it
75                 if ( iskanjeIzvedeno ) iskanjeIzvedeno = false
76             },
77             label = { Text("Ime države (npr. Slovenia) ") },
78             modifier = Modifier.fillMaxWidth()
79         )
80         Spacer( Modifier.height(8.dp))
81
82         Button(

```

```

82         Button(
83             onClick = {
84                 if ( iskalniNiz.isNotBlank() ) {
85                     iskanjeIzvedeno = true
86                     obseg.launch {
87                         nalaganje = true
88                         rezultati = dobiCene( drzava = iskalniNiz)
89                         nalaganje = false
90                     }
91                 }
92             },
93             modifier = Modifier.fillMaxWidth(),
94             enabled = iskalniNiz.isNotBlank()
95         ) {
96             Text("Poišči cene")
97         }
98
99         if ( nalaganja ) {
100             Box( Modifier.fillMaxWidth(), contentAlignment = Alignment.Center ) {
101                 CircularProgressIndicator()
102             }
103         }
104
105         if ( iskanjeIzvedeno && !nalaganje ) {
106             if ( rezultati.isNotEmpty() ) {
107                 Text("Zadečki:", style = MaterialTheme.typography.titleMedium)
108                 LazyColumn {
109                     items( rezultati ) { drzava ->
110                         Zadecek(
111                             podatki = drzava,
112                             obShrani = {
113                                 val zeObstaja = priljubljene.any { it.koda == drzava.koda }
114                                 if ( zeObstaja ) {
115                                     Toast.makeText( context = kon, text = "${drzava.ime} je že shranjena!", duration = Toast.LENGTH_SHORT ).show()
116                                 } else {
117                                     priljubljene = priljubljene + Shranjena(
118                                         drzava.ime, drzava.koda, drzava.valuta, drzava.enota,

```

```

119         drzava.cenaDizla, drzava.cenaBencina
120     )
121     shraniPriljubljene(kon, sez = priljubljene)
122     Toast.makeText(context = kon, text = "${drzava.ime} shranjena", duration = Toast.LENGTH_SHORT).show()
123 }
124 }
125 )
126 Spacer(Modifier.height(4.dp))
127 }
128 }
129 } else {
130     Box(Modifier.fillMaxWidth(), contentAlignment = Alignment.Center) {
131         Text("Država ni najdena", color = MaterialTheme.colorScheme.error)
132     }
133 }
134 }
135
136 Spacer(Modifier.height(16.dp))
137
138 Row(
139     Modifier.fillMaxWidth(),
140     horizontalArrangement = Arrangement.SpaceBetween,
141     verticalAlignment = Alignment.CenterVertically
142 ) {
143     Row(verticalAlignment = Alignment.CenterVertically) {
144         Text("Temni način")
145         Spacer(Modifier.width(8.dp))
146         Switch(checked = temniNacin, onCheckedChange = onTemniNacinChange)
147     }
148     Button(
149         onClick = {
150             obseg.launch {
151                 najcenejsaDrzava = najcenejsaDrzavaDizel()
152                 prikaziDialogNajcenejsa = true
153             }
154         },
155         colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.secondary)
156     ) {

```

```

157         Text("Najcenejši dizel")
158     }
159 }
160
161 Spacer(Modifier.height(8.dp))
162
163 Column(Modifier.weight(1f)) {
164     Text("Priljubljene države:", style = MaterialTheme.typography.titleMedium)
165     if (priljubljene.isEmpty()) {
166         Box(Modifier.fillMaxWidth(), contentAlignment = Alignment.Center) { Text("Ni shranjenih") }
167     } else {
168         LazyColumn {
169             items(priljubljene) { shran ->
170                 Priljubljene(
171                     podatki = shran,
172                     obOdstrani = {
173                         priljubljene = priljubljene - shran
174                         shraniPriljubljene(kon, sez = priljubljene)
175                         Toast.makeText(context = kon, text = "${shran.ime} odstranjena", duration = Toast.LENGTH_SHORT).show()
176                     },
177                     obOsvezi = {
178                         obseg.launch {
179                             val vse = dobivse()
180                             val posodobljena = vse.find { it.koda == shran.koda }
181                             if (posodobljena != null) {
182                                 priljubljene = priljubljene.map {
183                                     if (it.koda == shran.koda) it.copy(
184                                         cenaDizla = posodobljena.cenaDizla,
185                                         cenaBencina = posodobljena.cenaBencina,
186                                         valuta = posodobljena.valuta,
187                                         enota = posodobljena.enota
188                                     ) else it
189                                 }
190                                 shraniPriljubljene(kon, sez = priljubljene)
191                                 Toast.makeText(context = kon, text = "${shran.ime} osvežena", duration = Toast.LENGTH_SHORT).show()
192                             }
193                         }
194                     }
195                 )
196             }
197         }
198     }

```

```

195         )
196         Spacer(Modifier.height(8.dp))
197     }
198 }
199 }
200 }
201
202 Spacer(Modifier.height(16.dp))
203
204 Row(
205     Modifier.fillMaxWidth(),
206     horizontalArrangement = Arrangement.spacedBy(8.dp)
207 ) {
208     Button(
209         onClick = {
210             if (priljubljene.isNotEmpty()) {
211                 osvezevanjeVseh = true
212                 obseg.launch {
213                     val vse = dobiVse()
214                     val posodobljene = priljubljene.map { shran ->
215                         vse.find { it.koda == shran.koda }?.let {
216                             shran.copy(
217                                 cenaDizla = it.cenaDizla,
218                                 cenaBencina = it.cenaBencina,
219                                 valuta = it.valuta,
220                                 enota = it.enota
221                             )
222                         } ?: shran
223                     }
224                     priljubljene = posodobljene
225                     shraniPriljubljene(kon, sez = posodobljene)
226                     osvezevanjeVseh = false
227                     Toast.makeText(context = kon, text = "Vse države osvežene", duration = Toast.LENGTH_SHORT).show()
228                 }
229             }
230         },
231         modifier = Modifier.weight(1f),

```

```

232         enabled = priljubljene.isNotEmpty() && !osvezevanjeVseh
233     ) {
234         if (osvezevanjeVseh) {
235             Row {
236                 CircularProgressIndicator(Modifier.size(20.dp), strokeWidth = 2.dp)
237                 Spacer(Modifier.width(4.dp))
238                 Text("Osvežujem...")
239             }
240         } else Text("Osveži vse")
241     }
242
243     Button(
244         onClick = {
245             priljubljene = emptyList()
246             shraniPriljubljene(kon, sez = emptyList())
247             Toast.makeText(context = kon, text = "Vse počisteno", duration = Toast.LENGTH_SHORT).show()
248         },
249         modifier = Modifier.weight(1f),
250         colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)
251     ) {
252         Text("Počisti vse")
253     }
254 }
255 }
256
257
258 if (prikaziDialogNajcenejsa && najcenejsaDrzava != null) {
259     AlertDialog(
260         onDismissRequest = { prikaziDialogNajcenejsa = false },
261         title = { Text("Najcenejša država") },
262         text = {
263             Column {
264                 Text("${kodaVZastavo(najcenejsaDrzava!!.koda)} ${najcenejsaDrzava!!.ime}")
265                 najcenejsaDrzava!!.cenaDizla?.let {
266                     Text("Dizel: %.2f %s/%s".format(...args = it, najcenejsaDrzava!!.valuta, najcenejsaDrzava!!.enota))
267                 }
268                 najcenejsaDrzava!!.cenaBencina?.let {
269                     Text("Bencin: %.2f %s/%s".format(...args = it, najcenejsaDrzava!!.valuta, najcenejsaDrzava!!.enota))

```

```

270     }
271     },
272     confirmButton = {
273         Button(onClick = { prikaziDialogNajcenejsa = false }) { Text("Zapri") }
274     }
275 }
276 )
277 }
278 }
279
280 1 Usage
281 private suspend fun najcenejsaDrzavaDizel(): Bencinska? {
282     val vse = dobiVse()
283     return vse.filter { it.cenaDizla != null }
284         .minByOrNull { it.cenaDizla!! }
285 }
286
287 1 Usage
288 @Composable
289 fun Zadetek(podatki: Bencinska, obShrani: () -> Unit) {
290     Card(modifier.fillMaxWidth()) {
291         Row(modifier.padding(all = 12.dp), horizontalArrangement = Arrangement.SpaceBetween, verticalAlignment = Alignment.CenterVertically) {
292             Stolpec(podatki)
293             Button(onClick = obShrani) { Text("Shrani") }
294         }
295     }
296 }
297
298 1 Usage
299 @Composable
300 fun Priljubljen(podatki: Shranjena, obOdstrani: () -> Unit, obSvezi: () -> Unit) {
301     Card(modifier.fillMaxWidth()) {
302         Row(modifier.padding(all = 12.dp), horizontalArrangement = Arrangement.SpaceBetween, verticalAlignment = Alignment.CenterVertically) {
303             Stolpec(podatki, Modifier.weight(1f))
304             Row {
305                 IconButton(onClick = obSvezi) { Icon(imageVector = Icons.Default.Refresh, contentDescription = null) }
306                 Button(onClick = obOdstrani, colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)) {
307                     Text("Odstrani")
308                 }
309             }
310         }
311     }
312 }
313
314 2 Usages
315 @Composable
316 fun Stolpec(podatki: Any, modifier: Modifier = Modifier) {
317     Column(modifier = modifier) {
318         when (podatki) {
319             is Bencinska -> {
320                 Row(verticalAlignment = Alignment.CenterVertically) {
321                     Text(kodaVZastavo(podatki.koda), fontSize = MaterialTheme.typography.titleMedium.fontSize)
322                     Spacer(modifier.width(4.dp))
323                     Text(podatki.ime, style = MaterialTheme.typography.titleMedium)
324                 }
325                 Text("Celotna država", style = MaterialTheme.typography.bodySmall)
326                 podatki.cenaDizla?.let {
327                     Text("Dizel: %.2f %s/%s".format(it, podatki.valuta, podatki.enota))
328                 }
329                 podatki.cenaBencina?.let {
330                     Text("Bencin: %.2f %s/%s".format(it, podatki.valuta, podatki.enota))
331                 }
332             }
333             is Shranjena -> {
334                 Row(verticalAlignment = Alignment.CenterVertically) {
335                     Text(kodaVZastavo(podatki.koda), fontSize = MaterialTheme.typography.titleMedium.fontSize)
336                     Spacer(modifier.width(4.dp))
337                     Text(podatki.ime, style = MaterialTheme.typography.titleMedium)
338                 }
339                 Text("Celotna država", style = MaterialTheme.typography.bodySmall)
340                 podatki.cenaDizla?.let {
341                     Text("Dizel: %.2f %s/%s".format(it, podatki.valuta, podatki.enota))
342                 }
343                 podatki.cenaBencina?.let {
344                     Text("Bencin: %.2f %s/%s".format(it, podatki.valuta, podatki.enota))
345                 }
346             }
347         }
348     }
349 }

```